KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

# Hafta 5 Quiz

React II: Hooks, Form, Durum Paylaşımı

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

## Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

`AppIcerik`'teki belge başlığı efekti şu an `useEffect(() => {...}, [secilenKoltuklar.length])` biçiminde yazılmıştır. İkinci argümanı (bağımlılık dizisini) tamamen kaldırıp `useEffect(() => {...})` yazsaydınız ne olurdu?

(a) Efekt artık her render'dan sonra çalışırdı (`document.title` her seferinde yeniden atanırdı); ama bu efektin içinde bir `setState` çağrısı olmadığı için bu, sonsuz bir döngüye yol açmazdı — yalnızca gereksiz, tekrar eden bir atama olurdu.

(b) Efekt artık yalnızca ilk render'dan (mount) sonra bir kez çalışırdı; ikinci argümanın hiç verilmemesi boş dizi (`[]`) vermekle aynı anlama gelir.

(c) `tsc` bir derleme hatası verirdi; `useEffect`'in ikinci argümanı hiçbir zaman atlanamaz.

(d) Efekt hiç çalışmazdı; `useEffect` ikinci argüman verilmeden çağrıldığında React bu çağrıyı yok sayar.

## Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

`useGeriSayim` hook'undaki `useEffect`'in `return () => clearInterval(id);` satırını (temizleme fonksiyonunu) silerseniz ne olur?

(a) Hiçbir fark olmaz; React her `setInterval` çağrısını otomatik olarak izler ve bileşen ekrandan kalktığında (*unmount*) kendiliğinden temizler.

(b) `GeriSayim` bileşeni ekrandan kaldırılsa (*unmount*) bile zamanlayıcı arka planda çalışmaya devam eder, artık var olmayan bir bileşenin state'ini güncellemeye çalışır — bu bir bellek sızıntısına/konsol uyarısına yol açabilir.

(c) `tsc` bir derleme hatası verir; `useEffect`'in geri dönüş değeri olmadan kod derlenemez.

(d) Efekt bağımlılık dizisi (`[bittiMi]`) değiştiğinde eski zamanlayıcı React tarafından otomatik olarak durdurulur; temizleme fonksiyonu yalnızca kozmetik bir alışkanlıktır.

## Soru 3 (8 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki kesit `src/hooks/useGeriSayim.ts` dosyasından alınmıştır.

```
// baslangicSaniye'den geriye sayar: useState ile kalanSaniye tutulur,
// useEffect içindeki setInterval her saniye setKalanSaniye'yi çağırır.
// Süre bitince (bittiMi) zamanlayıcı YENİDEN kurulmaz; efekt her
```

## Soru 3 (devamı)

```tsx
// çalıştığında (ve bileşen ekrandan kalktığında) döndürdüğü temizleme
// fonksiyonuyla clearInterval çağrılır — bu olmasaydı bileşen unmount
// olsa bile zamanlayıcı arka planda çalışmaya devam ederdi.
export function useGeriSayim(baslangicSaniye: number) {
  const [kalanSaniye, setKalanSaniye] = useState(baslangicSaniye);
  const bittiMi = kalanSaniye <= 0;

  useEffect(() => {
    if (bittiMi) return;

    const id = setInterval(() => {
      setKalanSaniye((onceki) => (onceki <= 1 ? 0 : onceki - 1));
    }, 1000);

    return () => clearInterval(id);
  }, [bittiMi]);

  return { kalanSaniye, bittiMi };
}
```

(a) `bittiMi` `true` olduğunda `useEffect` içindeki `if (bittiMi) return;` satırı ne işe yarar — zamanlayıcı tekrar kurulur mu? (b) `kalanSaniye` 1'den 0'a düştüğü anda (bir sonraki `setInterval` tetiklemesinde) sırasıyla neler olur?

## Soru 4 (10 puan)

4 • Değerlendirme Hedef 1

`AppIcerik`'teki `document.title = ...` satırı neden doğrudan bileşenin `return` ifadesinden önceki bir satıra (render sırasında) değil, bir `useEffect` çağrısının içine yazılmıştır? Yan etki (*side effect*) kavramını ve vitrin tabelası benzetmesini kullanarak açıklayınız.

## Soru 5 (6 puan)

4 • Değerlendirme Hedef 2

`BiletFormu`'nda `ad` alanını güncellerken `setForm({ ...form, ad: e.target.value })` yerine `setForm({ ad: e.target.value })` (yayma operatörünü atlayarak) yazsaydınız ne olurdu?

(a) Hiçbir fark olmaz; `interface BiletFormVeri` yalnızca değişen alanı günceller, diğer alanları kendiliğinden korur.

(b) React formu otomatik olarak önceki değerlerle birleştirir; yayma operatörü yalnızca per-

## Soru 5 (devamı)

formans için, isteğe bağlı bir optimizasyondur.

(c) `tsc` bir tip hatası verir; `setForm` bir `BiletFormVeri` nesnesi bekler, ama `{ ad: string }` bu tipin `koltukSinifi` ve `kampanyaOnay` alanlarını içermez — nesne bu tipe uymaz.

(d) Yalnızca `koltukSinifi` alanı sıfırlanır, `kampanyaOnay` bundan etkilenmez.

## Soru 6 (6 puan)

4 • Değerlendirme Hedef 2

`kampanyaOnay` alanı için `<input type="checkbox" checked={form.kampanyaOnay} onChange={...} />` yazılmıştır. `checked` yerine `value={form.kampanyaOnay}` yazsaydınız ne olurdu?

(a) Kod sorunsuz derlenir; `checkbox`'ın işaretli görünüp görünmediği yine React state'inden takip edilir, sonuç birebir aynıdır.

(b) React, `checkbox` öğeleri için `value`'yu otomatik olarak `checked`'e çevirir; ikisi arasında pratikte fark yoktur.

(c) Tarayıcı bu `checkbox`'ı otomatik olarak `disabled` yapar; kullanıcı kutuyu hiç işaretleyemez.

(d) `tsc` bir tip hatası verir; `value` özniteliği `string`/`number`/dizi bekler, `form.kampanyaOnay` ise bir `boolean`'dır — bu tip bu özniteliğe atanamaz.

## Soru 7 (8 puan)

4 • Değerlendirme Hedef 2

Aşağıdaki kesit `src/bilesenler/BiletFormu.tsx` dosyasından alınmıştır.

```
// İlk satır e.preventDefault(): aksi halde tarayıcı VARSAYILAN
// davranışıyla sayfayı yeniden yükler ve tüm React state'i sıfırlanır.
// Sonra iki koşul kontrol edilir (ad boş mu, sepette hiç koltuk yok
// mu); koşullardan biri sağlanmazsa hata state'i doldurulur ve Bilet[]
// OLUŞTURULMAZ.
function submitEt(e: FormEvent<HTMLFormElement>) {
  e.preventDefault();

  if (form.ad.trim() === '') {
    setHata('Ad alanı boş olamaz.');
    return;
  }
  if (secilenKoltuklar.length === 0) {
    setHata('Sepette en az bir koltuk seçili olmalı.');
    return;
  }

  const satinAlmaZamani = new Date().toISOString();
  const yeniBiletler: Bilet[] = secilenKoltuklar.map((koltuk) => ({
    id: sonrakiBiletId++,
    koltukId: koltuk.id,
    kullaniciAdi: form.ad,
    satinAlmaZamani,
    durum: 'olusturuldu',
  }));

```

## Soru 7 (devamı)

```
27   onBiletlerOlustu(yeniBiletler);
28   sepetiTemizle();
29   setForm(BOS_FORM);
30   setHata(null);
31 }
```

(a) Kullanıcı `ad` alanına yalnızca boşluk karakterleri (`"  "`) girip formu gönderirse ne olur, hangi satır bunu yakalar? (b) Sepette 2 koltuk seçiliyken `ad` alanı doğru doldurulup form gönderildiğinde, `submitEt` fonksiyonunun çağırdığı tüm state güncellemelerini/yan çağrıları sırasıyla sayınız.

## Soru 8 (10 puan)

4 • Değerlendirme Hedef 2

`submitEt` fonksiyonunun ilk satırının neredeyse her zaman `e.preventDefault()` olması gerektiğini biliyorsunuz. Bu satırı yazmasaydınız ne olurdu? Bunu Hafta 1'in HTTP istek/cevap bilgisiyle ilişkilendirerek açıklayınız.

## Soru 9 (6 puan)

4 • Değerlendirme Hedef 3

Bir geliştirici yanlışlıkla `<GeriSayim />`'i `App` bileşeninin `SepetProvider` sarmalayıcısının dışına yerleştirseydi ne olurdu?

(a) `useSepet()` içindeki `if (!baglam) throw new Error(...)` satırı çalışır; `"useSepet() yalnızca SepetProvider içinde kullanılabilir."` mesajıyla açık bir hata fırlatılır — sorunun kaynağı hemen, anlaşılır biçimde ortaya çıkar.

(b) `useContext(SepetContext)` bileşen ağacının herhangi bir yerindeki `SepetProvider`'ı otomatik olarak bulur; render ağacındaki konum önemli değildir.

(c) `tsc` bu yanlış yerleştirmeyi derleme zamanında tespit edip kodun derlenmesini engeller.

(d) `baglam` `undefined` döner, `GeriSayim` sessizce boş bir `<div>` render eder; hiçbir hata/uyarı görünmez.

## Soru 10 (6 puan)

4 • Değerlendirme Hedef 3

`SepetContext`'teki `koltukSec` fonksiyonu, zaten sepette (`secilenKoltuklar` içinde) olan bir koltuğa tekrar tıklandığında ne yapar?

## Soru 10 (devamı)

(a) Koltuğu diziye ikinci kez ekler; `toplamTutar` bu koltuk için çift sayılır.

(b) Hiçbir şey yapmaz; bir koltuk sepete eklendikten sonra yalnızca `BiletFormu` gönderildiğinde sepetten çıkarılabilir.

(c) Koltuğu `secilenKoltuklar` dizisinden `filter` ile çıkarır; `setSecilenKoltuklar` yeni bir dizi referansıyla çağrılır.

(d) `secilenKoltuklar.push(koltuk)` ile diziyi doğrudan mutasyona uğratıp aynı referansı korur.

## Soru 11 (8 puan)

4 • Değerlendirme Hedef 3

Aşağıdaki kesitler `src/context/SepetContext.tsx` ve `src/bilesenler/Koltuk.tsx` dosyalarından alınmıştır.

```tsx
// Seçili koltukları tek bir useState ile tutar; koltukSec aynı koltuk
// zaten sepetteyse çıkarır, değilse ekler (değişmezlik ilkesiyle her
// zaman YENİ bir dizi üretir). toplamTutar her render'da
// secilenKoltuklar'dan yeniden hesaplanır, ayrıca saklanmaz.
export function SepetProvider({ biletFiyati, children }: SepetProviderProps) {
  const [secilenKoltuklar, setSecilenKoltuklar] = useState<Koltuk[]>([]);

  function koltukSec(koltuk: Koltuk) {
    setSecilenKoltuklar((onceki) =>
      onceki.some((k) => k.id === koltuk.id)
        ? onceki.filter((k) => k.id !== koltuk.id)
        : [...onceki, koltuk],
    );
  }

  function sepetiTemizle() {
    setSecilenKoltuklar([]);
  }

  const deger: SepetBaglami = {
    secilenKoltuklar,
    toplamTutar: secilenKoltuklar.length * biletFiyati,
    koltukSec,
    sepetiTemizle,
  };

  return <SepetContext.Provider value={deger}>{children}</SepetContext.Provider>;
}
```

```tsx
// Koltuğun seçili olup olmadığı artık kendi yerel state'inde değil,
// ortak SepetContext'teki secilenKoltuklar dizisinde tutulur; App/
// KoltukPlani'nın haberi olmadan bu bileşen doğrudan Context'ten okur.
// Tıklama koltukSec'i çağırıp koltuğu sepete ekler/çıkarır; dolu bir
// koltuk (veri.ts'ten gelen kalıcı durum) hiç tıklanamaz.
const { secilenKoltuklar, koltukSec } = useSepet();
const seciliMi = secilenKoltuklar.some((k) => k.id === koltuk.id);
const durum = koltuk.durum === 'dolu' ? 'dolu' : seciliMi ? 'secili' : 'bos';

function tikla() {
  if (koltuk.durum === 'dolu') return;
```

## Soru 11 (devamı)

```
12   koltukSec(koltuk);
13 }
```

(a) `'dolu'` durumundaki bir koltuğa tıklandığında `koltukSec` çağrılır mı? Neden? (b) `toplamTutar` Neden ayrı bir `useState` olarak saklanmaz, her render'da `secilenKoltuklar`'dan yeniden hesaplanır — bunun avantajı nedir?

## Soru 12 (10 puan)

4 • Değerlendirme Hedef 3

`SepetContext` Kullanmadan, `App`'teki sepet bilgisini (`secilenKoltuklar`, `koltukSec`) yalnızca `props` ile `App` → `KoltukPlani` → `Koltuk` zincirinde taşımak isteseydiniz, `KoltukPlani` bileşeninin `props` tanımında ne değişirdi ve bu yaklaşımın `SepetContext`'e göre dezavantajı ne olurdu? Bina anons/santral sistemi benzetmesini kullanarak açıklayınız.

## Soru 13 (6 puan)

4 • Değerlendirme Hedef 4

Her bileşende `useContext(SepetContext) + if (!baglam) throw ...` yazmak yerine `useSepet()` özel hook'unu kullanmanın asıl kazancı nedir?

(a) `useSepet()` Context'i daha hızlı okur; `useContext`'in doğrudan çağrılması ekstra bir render gecikmesine yol açar.

(b) `null` kontrolü ve hata mesajı tek bir yerde (`useSepet` içinde) yazılır; `Koltuk`, `SepetOzeti`, `BiletFormu`, `GeriSayim` gibi her bileşen bu 2-3 satırı tekrar tekrar yazmak zorunda kalmaz.

(c) `useSepet()` kullanmak `SepetProvider`'ı kurmayı gereksiz kılar; Context artık bir Provider olmadan da çalışabilir hale gelir.

(d) `useSepet()` bir bileşen değil bir fonksiyon olduğu için JSX döndürebilir; bu sayede diğer bileşenlerin görünümünü de paylaşır.

## Soru 14 (8 puan)

4 • Değerlendirme Hedef 4

Aşağıdaki kesitler `src/hooks/useSepet.ts` ve `src/bilesenler/GeriSayim.tsx` dosyalarından alınmıştır.

## Soru 14 (devamı)

```
// useContext(SepetContext)'i sarar; her bileşenin kendi başına
// useContext + null kontrolü yazması yerine yalnızca useSepet() çağırması
// yeter. baglam null dönerse (SepetProvider dışında kullanılmışsa)
// belirsiz bir çalışma zamanı hatası yerine AÇIK, anlaşılır bir hata
// fırlatır.
export function useSepet() {
  const baglam = useContext(SepetContext);
  if (!baglam) {
    throw new Error('useSepet() yalnızca SepetProvider içinde kullanılabilir.');
  }
  return baglam;
}
```

```
// useGeriSayim(120) kalan saniyeyi/bittiMi'yi okur; süre bittiğinde
// sepetiTemizle() çağıran ayrı bir efekt kurar — böylece kullanıcı
// süresi içinde formu göndermezse sepet kendiliğinden boşalır.
const { kalanSaniye, bittiMi } = useGeriSayim(BASLANGIC_SANIYE);
const { sepetiTemizle } = useSepet();

useEffect(() => {
  if (bittiMi) sepetiTemizle();
}, [bittiMi]);
```

(a) `useSepet.ts` içindeki `throw` satırı silinseydi ve `GeriSayim`, `SepetProvider`'ın dışında kullanılsaydı, `const { sepetiTemizle } = useSepet();` satırında hangi tür bir hata oluşurdu? (b) `GeriSayim`'deki `useEffect`'in bağımlılık dizisi neden yalnızca `[bittiMi]`'dir — neden `secilenKoltuklar` ya da `kalanSaniye` değildir?

## Soru 15 (10 puan)

4 • Değerlendirme Hedef 4

`useSepet` ve `useGeriSayim`'i ayrı dosyalarda (`src/hooks/`) tutmak yerine, bu mantığı doğrudan `Koltuk`, `SepetOzeti`, `BiletFormu`, `GeriSayim` bileşenlerinin içine kopyala-yapıştır yapsaydınız ne kaybederdiniz? Bileşen bölme ölçütlerinden en az ikisini kullanarak açıklayınız; el kitabı benzetmesini kullanabilirsiniz.